CS301: Advanced Operating Systems - Lecture Notes

Unit 1: Virtual Memory & Page Replacement Algorithms

Virtual memory decouples the programmer's view of memory from physical reality. Each process runs in its own private 64-bit address space divided into fixed-size units called pages (typically 4 KB or 2 MB huge pages). The MMU (Memory Management Unit) translates virtual addresses to physical frame numbers using a multi-level page table structure.

Key replacement policies include:
1. Optimal Replacement (OPT): Discard the page that will not be accessed for the longest period in the future (theoretical baseline).
2. Least Recently Used (LRU): Evict the page that has not been referenced for the longest duration. Uses timestamp or hardware reference bits.
3. Clock / Second Chance: An approximation of LRU using a circular buffer and a use-bit.

Thrashing occurs when the sum of working set sizes across all active processes exceeds physical RAM, causing constant page faults.

CS301: Advanced Operating Systems - Lecture Notes

Unit 2: Concurrency, Mutexes, and Deadlock Prevention

Concurrency introduces race conditions when two or more threads attempt to manipulate shared memory concurrently.

To ensure mutual exclusion, systems utilize:
- Mutex Locks: Binary flag ensuring only one thread enters the critical section at a time.
- Counting Semaphores: Maintained via P() [wait] and V() [signal] operations, allowing up to N concurrent threads.
- Read-Write Locks: Permits multiple concurrent readers or a single exclusive writer.

Deadlock requires four necessary conditions (Coffman Conditions):
1. Mutual Exclusion
2. Hold and Wait
3. No Preemption
4. Circular Wait

Prevention: Impose a global total ordering on all locks (Lock Acquisition Hierarchy) to break circular wait.

CS301: Advanced Operating Systems - Lecture Notes

Unit 3: Distributed Consensus & The Raft Protocol

Consensus in distributed systems requires multiple independent nodes to agree on state transitions despite node crashes or network partitions.

The Raft consensus algorithm decomposes consensus into three subproblems:
1. Leader Election: When a node's heartbeat timer expires without contact from the leader, it increments term number, votes for itself, and broadcasts RequestVote RPCs.
2. Log Replication: The elected leader receives client state machine commands, appends them to its local write-ahead log, and broadcasts AppendEntries RPCs to followers.
3. Safety Invariant: Once an entry has been committed in a given term, that entry will be present in the logs of the leaders for all higher terms.

Quorum requirement: (N/2) + 1 active nodes must acknowledge an entry before it is considered committed.